High-Confidence Computing 6 (2026) 100351

Contents lists available at ScienceDirect

High-Confidence Computing

journal homepage: www.sciencedirect.com/journal/high-confidence-computing

Research article

# LwRustIP: Memory-safe and efficient embedded networking stack with ownership semantics

Guangyong Shang [a,b], Guangpeng Qi [b], Jianing Ren [c], Xianqi Jin [b], Wanjiang Shen [b], Junchao Li [a], Runyu Pan [a,*]

[a] School of Computer Science and Technology, Shandong University, Qingdao 266237, China
[b] Inspur Yunzhou industrial Internet Co., Ltd., Jinan 250101, China
[c] HighGo Software Co., Ltd., Jinan 250101, China

ARTICLE INFO

*Article history:*
Received 1 June 2025
Revised 19 July 2025
Accepted 27 August 2025
Available online 9 September 2025

*Keywords:*
Embedded network protocol stack
Memory safety
Zero-copy architecture
Rust ownership semantics

ABSTRACT

As modern embedded systems are increasingly network connected, their protocol stacks expose themselves as a surface that is frequently attacked. While C-based implementations such as *LwIP* are efficient, their lack of memory safety induces critical vulnerabilities such as buffer overflows, dangling pointers, and use-after-free, leading to remote code execution or privilege escalation. In this paper, we present LwRustIP, a memory-safe embedded networking stack reimplemented in Rust and compatible with *LwIP*. We also share our development experience. LwRustIP replaces unsafe linked-list memory management with a custom allocator that honors the Rust ownership semantics, leverages zero-copy techniques for inter-layer packet handoffs, and applies lock-free object pools for concurrent buffer management. These design choices ensure memory safety while maintaining performance comparable to traditional C-based implementations. We deploy LwRustIP on ARM-based embedded platforms and evaluate its correctness, performance, and memory safety. Experimental results show that LwRustIP achieves memory safety without incurring measurable performance overhead compared to the original C-based implementation. Our experience highlights the practical challenges and benefits of using Rust for low-level system components and offers guidance for future efforts in memory-safe reengineering of legacy C codebases.



## 1. Introduction

With the rise of the Internet of Things (IoT), networking has become a fundamental feature of modern embedded systems. However, the integration of networking components has significantly expanded the attack surface of embedded platforms [1–3], exposing critical vulnerabilities that threaten system reliability and security [4,5]. This is caused by the inherent lack of pointer discipline and memory safety in the C programming language, which is the prime choice of implementation for embedded networking stacks due to its performance and simplicity. As statistics suggest, over 68% of Common Vulnerabilities and Exposures (CVEs) [6] in embedded networking originate from memory safety violations such as buffer overflows, dangling pointers, double-free and use-after-free [7,8]. Several examples that compromise confidentiality, integrity, or availability are listed in Table 1, though the list is not exhaustive. Although these CVEs have been patched, new vulnerabilities continue to emerge in newer versions of the stacks due to the fundamental lack of compile-time memory safety checks in the C language.

The Rust programming language offers a compelling solution to systemic memory safety issues. As a modern systems programming language, Rust ensures memory safety without relying on garbage collection, provides concurrency safety by preventing data races, and supports zero-cost abstractions that compile into efficient machine code. Its ownership model enforces strict aliasing rules and guarantees safe memory management at compile time, effectively eliminating entire classes of memory-related bugs common in C programs. Additionally, Rust offers fine-grained control over system resources, making it particularly suitable for performance-critical and resource-constrained environments such as embedded systems.

Despite its advantages, rewriting or porting low-level system components to Rust is a non-trivial task. Embedded network stacks written in C often depend on global state, intrusive linked data structures, manual memory management, and unsafe concurrency practices—all of which conflict with Rust's ownership and lifetime rules. Addressing these challenges necessitates structural redesigns and a fundamental rethinking of legacy code architectures.

* Corresponding author.
*E-mail address:* rypan@sdu.edu.cn (R. Pan).

https://doi.org/10.1016/j.hcc.2025.100351

**Table 1**
CVE-listed memory safety vulnerabilities in embedded network stacks.

| CVE ID | Description |
|---|---|
| CVE-2024–7490 | An input validation flaw in the DHCP server implementation of the *LwIP* stack can lead to a buffer overflow, enabling attackers to execute arbitrary code via specially crafted packets. |
| CVE-2020–13985 | The RPL extension header parsing function in uIP-Contiki-OS fails to validate unsafe integer conversions during header field extraction, allowing memory corruption via maliciously crafted headers. |
| CVE-2020–13986 | The RPL extension header processing in uIP-Contiki-OS lacks proper validation of length fields in received headers, allowing attackers to trigger infinite loop conditions through oversized values. |
| CVE-2020–13987 | A boundary validation flaw in TCP/UDP packet parsing across open-iscsi, uIP-Contiki-OS, and uIP allows out-of-bounds memory reads during checksum calculation, caused by the acceptance of arbitrary header lengths. |
| CVE-2020–17442 | picoTCP-NG and picoTCP fail to validate the IPv6 header length fields, allowing attackers to induce infinite processing loops by injecting packets with malformed length values. |
| CVE-2020–17468 | Memory corruption vulnerabilities in FNET's handling of IPv6 Hop-by-Hop extension headers arise from unchecked option length parameters, allowing adversarial manipulation of heap memory structures. |
| CVE-2020–25110 | Nut/Net fails to validate length bytes during DNS domain name parsing, exposing the system to denial-of-service (DoS) or remote code execution (RCE) via buffer overflows. |
| CVE-2021–26788 | Improper input sanitization in Oryx Embedded CycloneTCP enables resource exhaustion attacks, resulting in prolonged denial-of-service (DoS) conditions. |
| CVE-2023–38562 | A double-free vulnerability in Weston Embedded uC/TCP-IP leads to memory corruption through improper deallocation of network buffers, potentially destabilizing system operations. |
| CVE-2023–5941 | In FreeBSD, unchecked system call return values cause heap buffer overflows, enabling attackers to overwrite adjacent memory regions. |
| CVE-2019–12257 | A buffer overflow in VxWorks' IPNet stack allows attackers to overwrite critical stack memory via crafted network packets, resulting in arbitrary code execution. |
| CVE-2022–38371 | Nucleus Net suffers from persistent memory leaks due to improper resource deallocation during socket termination, leading to gradual exhaustion of system memory. |

In this paper, we present LwRustIP, a memory-safe reimplementation of the *LwIP* in Rust. Unlike prior work that introduces novel protocol designs or abstractions, our objective is pragmatic: to explore and document the practical experience of porting a mature, performance-critical system module from C to Rust. LwRustIP preserves the functional behavior and performance characteristics of the original *LwIP* stack while eliminating its memory safety vulnerabilities.

Our implementation integrates three key techniques:

- A hybrid memory allocator that combines fixed-size block pools (64B–2KB) for protocol headers with dynamically allocated, linked payload buffers. This design enables O(1) memory allocation with minimal fragmentation.
- Cross-layer zero-copy packet transfer, achieved through Rust's compile-time validated mutable references, which enforces safe buffer access while eliminating unnecessary data copies between protocol layers.
- Protocol state machines with type-safe packet structures, which incorporate boundary validation and byte-order correctness, reducing runtime errors during packet parsing and processing.

**Contributions.** The primary contribution of this paper is an experience report demonstrating that memory safety can be retrofitted into legacy embedded systems without compromising performance or compatibility.

- We demonstrate how Rust memory safety features **eliminate existing memory vulnerabilities** by reimplementing *LwIP* in Rust, resulting in a safe yet efficient networking stack.
- We detail the migration process, emphasizing the **engineering challenges and lessons learned**, offering practical reference for future C-to-Rust transitions.
- We evaluate LwRustIP on ARM-based embedded platforms and compare it with *LwIP* and Linux. Results show that **memory safety is achieved without incurring additional performance overhead**.

This paper systematically reimplements the *LwIP* in Rust, offering insights into the feasibility and engineering practices of incorporating memory safety without compromising performance or compatibility.

## 2. Related work

The transition from C to Rust in system-level software development has garnered significant attention due to Rust's promise of memory safety without sacrificing performance. This section reviews Rust's application in operating systems, embedded systems, and network protocols; examines literature on memory safety; discusses secure coding standards; and highlights vulnerabilities in embedded network stacks.

### 2.1. Rust for systems and memory safety

Rust has seen increasing adoption in systems programming due to its safety and performance guarantees. Tock OS [9], Redox OS [10], Drone OS [11] and Theseus OS [12] explore Rust-based kernels for embedded or general-purpose systems.

In the networking domain, smoltcp [13], embassy-net [14], and rusty [15] demonstrate TCP/IP stacks implemented in Rust for resource-constrained or async embedded environments. Rust-based drivers and RTOS components [16,17] further illustrate Rust's suitability for low-level embedded development.

Other projects like rustix [18], librsvg [19], and Firecracker [20] reflect the growing trend of partial or full Rust rewrites in system-level components to enhance security and maintainability.

The RustBelt project [21] formally proves the soundness of Rust's type and ownership system, while the Rust's Unsafe Code Guidelines [22] clarify safety invariants in unsafe contexts. Verification tools such as Prusti [23], Kani [24], Creusot [25], and MIRAI [26] support formal and symbolic analysis of Rust code. These works collectively show that memory safety can be achieved even in low-level system programming without sacrificing expressiveness.

### 2.2. Secure coding standards in C/C++

Legacy C/C++ systems often adopt secure coding guidelines — such as the CERT C Secure Coding Standard [27], JSF C++[28], and MISRA C [29] — to mitigate memory-related errors. Similar practices are recommended for high-assurance embedded systems [30]. However, these standards are largely advisory and not enforced by compilers, which limits their ability to prevent critical vulnerabilities such as buffer overflows, dangling pointers, and type confusion.

### 2.3. Embedded TCP/IP stacks

Several lightweight TCP/IP stacks have been developed to support embedded and resource-constrained systems. Notable examples include *LwIP* [31], uIP [32], and FNET [33], which prioritize small memory footprints and basic protocol compliance. *LwIP* is widely adopted in IoT devices and RTOS-based platforms due to its modularity and configurability. uIP, designed for 8-bit and 16-bit microcontrollers, offers a minimal TCP/IP stack implementation. The FNET is a free, open-source, dual TCP/IPv4 and IPv6 stack designed for building embedded communication software on 32-bit MCUs.

However, these stacks are predominantly implemented in C. C-based embedded protocol stacks are frequently affected by memory corruption bugs, stemming from C's lack of memory safety guarantees and the challenges of validating complex stateful logic in embedded network protocols. Despite their operational efficiency, these stacks remain vulnerable due to manual memory management and unsafe pointer manipulation.

Notable disclosures include AMNESIA:33 [5] and URGENT/11 [4], which revealed dozens of CVEs including heap overflows, double free, use-after-free, and memory leak vulnerabilities [34–37].

### 2.4. Rust-based reengineering of legacy systems

smoltcp [13] is a minimalist, event-driven TCP/IP stack written in Rust, designed for bare-metal and real-time systems. Its heapless, non-blocking, and statically configured architecture suits resource-constrained environments, but omits features like congestion control and full DNS, limiting broader applicability. In contrast, LwRustIP reimplements the LwIP stack with Rust's ownership model, supporting a complete protocol suite (ARP, ICMP, UDP, TCP), dynamic interface management, and modular refactoring to ensure memory safety and compatibility.

RustPython [38], a Rust-based reimplementation of the Python 3 interpreter, further exemplifies Rust's suitability for reengineering complex runtime systems. Like this paper, it addresses the trade-off between safety and performance, demonstrating the feasibility of replacing legacy C infrastructure with safe, efficient Rust alternatives.

## 3. LwRustIP design

The design of LwRustIP is guided by the original *LwIP*, a widely adopted legacy embedded network stack, which we reimplemented in Rust to leverage Rust's memory safety features. This process required a fundamental rethinking of the architecture, state management, and interface boundaries due to Rust's strict type system and ownership semantics. In this section, we detail the key design strategies that preserve *LwIP*'s performance while conforming to Rust's language constraints.

The migration was guided by three core principles:

- Memory Safety First: Replace all raw pointer manipulations and manual memory management with safe abstractions that enforce invariants at compile time.
- Idiomatic Rust: Favor Rust's type system, trait abstractions, and modular design over direct C-to-Rust translation. The code should be maintainable by Rust developers, not merely a faithful reproduction of C semantics.
- Scoped Unsafe Usage: Restrict unsafe code to well-defined, low-level contexts — such as hardware interaction, C FFI, and device driver access — while maintaining safe abstractions throughout the rest of the system.

### 3.1. Fine-grained state management and safe initialization

In *LwIP* critical runtime states such as the TCP connection table, ARP cache, and network interface configurations are maintained as mutable global variables. This design is inherently unsafe and non-reentrant, relying on a single global mutex to guard access, which limits concurrency and complicates safety reasoning.

In our Rust-based implementation, we decouple these global states into independently managed singleton modules, each protected by fine-grained synchronization primitives (e.g., Mutex or RwLock). Unlike *LwIP*'s coarse-grained locking, our design ensures that only one global state is accessed per critical section, enabling greater concurrency while maintaining safety. Temporary caching is employed to reduce lock contention and minimize the overhead associated with frequent lock acquisitions.

This modular encapsulation eliminates the need for unsafe global mutability, enables thread-safe access, and enhances testability and maintainability by isolating protocol logic into independently verifiable components.

### 3.2. Safe containers and ownership-aware access

*LwIP* relies heavily on custom intrusive linked lists, implemented via structure embedding and raw pointers, to manage timer events, packet buffers, and transmission queues. While this approach is effective in C, it is incompatible with Rust's strict ownership and borrowing rules, which prohibit aliasing mutable references and lack built-in support for safe intrusive data structures.

To address this, we replace low-level linked lists with safe, ownership-aware containers wherever possible. In scenarios that demand efficient removal or traversal without violating Rust's ownership rules — such as retransmission buffers — we adopt specialized crates that leverage lifetime tracking to ensure memory safety.

For cases requiring persistent references to list elements, we redesign access patterns to use stable indices or opaque identifiers instead of raw pointers.

This shift from a pointer-centric design to ownership-driven abstractions prioritizes memory safety, improves code clarity, and enhances long-term maintainability, at the cost of some low-level flexibility inherent to C.

### 3.3. Smart pointers with explicit lifetimes

C code in *LwIP* relies heavily on raw pointers to manage protocol control blocks (PCBs), buffer chains, and shared memory structures. In our Rust implementation, these constructs are replaced with a combination of smart pointers and lifetime annotations that enforce memory safety and ownershipsemantics at compile time:

- Box<T>is used for exclusive heap allocation;
- Rc<T>and Arc<T>are used for shared ownership in single- and multi-threaded contexts, respectively;
- RefCell<T>or Mutex<T>provide interior mutability where required;
- Pin<Box<T>>is used to ensure stable memory addresses for self-referential data structures.

Certain data structures, such as TCP PCBs, contain cyclic or bidirectional references. As Rust's ownership model does not naturally support such structures, we introduce reference-counted smart pointers and use Weak<T>links to break strong reference cycles where applicable.

### 3.4. Trait-based layer composition

*LwIP* models network packets as deeply nested C structs, which is inflexible in Rust due to strict ownership and lifetime constraints.

In LwRustIP, we redesign packet parsing and emission using a compositional, trait-based approach. Each protocol layer (Ethernet, IP, TCP, UDP) implements the following traits:

- Parseable: Parses headers from a byte slice.
- Serializable: Emits headers into a mutable byte buffer.
- Header: Represents parsed header fields.

This enables zero-copy parsing, layer decoupling, and flexible extension (e.g., support for IPv6 or new transport protocols).

Protocol handlers operate over buffer views without assuming deep structural embedding, enabling layer-by-layer parsing and reconstruction, independent unit testing of each protocol parser, and efficient header construction without unnecessary memory copying.

### 3.5. Layered safety: Isolating unsafe interfaces in the HAL

In LwRustIP, we enforce a strict separation between safe and unsafe code by organizing the system into distinct layers. The lower layers — most notably the hardware abstraction layer (HAL), which interfaces with C-based drivers and hardware-specific routines — inevitably involve the use of unsafe code to support operations such as raw pointer manipulation, memory-mapped I/O, and DMA buffer coordination.

These unsafe operations are strictly confined to scoped, low-level modules, which expose only safe, documented APIs to the rest of the system. Higher layers of the stack, including protocol logic and state management, are implemented entirely in safe Rust and interact with the HAL through these safe abstractions.

This design preserves Rust's safety guarantees in the majority of the codebase while pragmatically supporting the requirements of low-level systems programming. By localizing unsafety to a small, controlled boundary, we ensure that most of the system remains memory-safe, testable, and easier to reason about.

## 4. LwRustIP implementation

### 4.1. Architecture refactoring and protocol component implementation

To address the safety, modularity, and maintainability challenges of *LwIP*'s original design, our Rust reimplementation performs a comprehensive architectural refactoring. The new design strictly enforces ownership boundaries, eliminates implicit state coupling, and leverages Rust's type system and trait abstractions for clean protocol layer separation. Compared to *LwIP*'s macro-heavy, global-state-driven C codebase, our system adopts a component-oriented and memory-safe architecture from the ground up.

To enable a memory-safe and modular Rust implementation, we restructure the network stack into a set of explicitly owned and independently testable components. Each major subsystem is encapsulated within a dedicated Rust module (crate or submodule), with well-defined responsibilities, ownership semantics, and interface boundaries.

#### 4.1.1. Modular protocol managers

Each network protocol or subsystem is encapsulated as a self-contained Rust module, organized around a central manager struct that maintains internal state, drives protocol logic, and defines the public interface for that component. This modular architecture enforces separation of concerns, facilitates precise reasoning about protocol behavior, and enables independent development and testing of individual components.

Rather than relying on mutable global variables or shared control structures, each manager module maintains exclusive ownership of its associated state, enforced by Rust's ownership and borrowing semantics. This design not only prevents data races and aliasing but also clearly defines responsibility for resource cleanup and state transitions.

- `TcpManager`: Responsible for the full lifecycle of TCP connections, including creation, teardown, and state transitions (e.g., SYN-RECEIVED to ESTABLISHED). It manages retransmission queues, per-connection timers, and buffer management. Each active connection is stored in a HashMap keyed by a unique socket ID, ensuring isolation and explicit ownership.
- `UdpManager`: Maintains a table of active UDP sockets and routes inbound datagrams to matching handlers based on destination port. Since UDP is connectionless, the manager focuses on socket state, buffer delivery, and basic multiplexing logic.
- `NetIfManager`: Manages the lifecycle of all network interfaces (e.g., Ethernet, loopback). It handles packet transmission and reception, encapsulates device driver interactions, and implements routing logic to direct outbound packets to the appropriate interface.

Each manager exposes a minimal, well-defined public API with safe method signatures such as `accept()`, `recv()`, `transmit()`, or `resolve()`. No internal mutable state, raw pointers, or callback hooks are exposed to consumers. This strict encapsulation minimizes the risk of misuse, simplifies maintenance, and aligns with Rust's idioms of safe abstraction.

This manager-centric, reified design establishes a clean, modular, and memory-safe foundation for implementing complex protocol logic, while maintaining clarity and ease of maintenance across the system's architectural layers.

**Lessons Learned**

Our initial approach mimicked *LwIP*'s C-style architecture using raw pointers and callbacks, which clashed with Rust's ownership model. Refactoring to protocol-specific managers with explicit ownership and safe interfaces highlighted the importance of embracing Rust's type system and modular design principles for building maintainable and safe systems.

#### 4.1.2. Global state representation: Safe and flexible alternatives to LwIP-style globals

Unlike the C-based *LwIP*, which heavily depends on static global variables and external linkage to share subsystem state, our Rust implementation adopts a diversified and safety-oriented approach to global state management. While global state in C offers simplicity, it is prone to data races, implicit dependencies, and initialization order bugs. In contrast, Rust enforces stringent guarantees on global variable access, initialization, and mutation, fostering a more disciplined and robust design. To accommodate various functional requirements, we employ tailored global state management patterns adapted to specific use cases:

- **Const**: For immutable compile-time values, Rust provides `const`. These are inlined and have no runtime overhead.

- **Static Variables (Discouraged)**: Rust supports `static mut` for mutable global variables, but they require `unsafe` access and are not thread-safe. Due to safety concerns, we avoid using this mechanism.
- **Atomic Globals for Counters and Flags**: For lightweight global state such as counters or flags, `Atomic` provides a lock-free, thread-safe primitive. It is well-suited for count or state tracking.
- **Lazy Initialization with `Lazy` and `Mutex`**: For more complex global state (e.g., protocol managers), we use `once_cell::sync::Lazy` to enable thread-safe, runtime-initialized statics. Each manager is wrapped in a `Mutex` to enable safe interior mutability across threads. Accessing such a global safely requires calling `lock()` and handling the guard object correctly. The mutex guard grants temporary access without transferring ownership and automatically releases the lock when it goes out of scope. Misuse, such as nested locking, may lead to deadlocks.

While preserving the conceptual model of centralized subsystems, this layered strategy eliminates the hazards of unguarded shared state. It allows fine-grained testability, runtime configuration, and memory safety—capabilities that static C globals fundamentally lack. Our Rust design encourages encapsulation, thread safety, and modular control over subsystem lifecycles.

**Lessons Learned**

Porting from C to Rust revealed fundamental differences in global state management. Our initial use of static mut in Rust encountered borrow checker conflicts and safety issues. Rust enforces stricter rules which require explicit ownership and synchronization. This shift emphasized the need global state redesign.

*4.1.3. Trait-based packet parsing and compositional protocol stacks*

Rather than relying on C-style packed structs and unsafe memory casting for protocol header parsing and construction, we adopt a trait-oriented design that cleanly separates parsing logic, serialization, and field access. Each protocol defines a set of traits that encapsulate its behavior over raw packet data:

- `Parseable`: Converts a byte slice (`&[u8]`) into a structured packet type. Parsing errors (e.g., insufficient length or invalid checksums) are explicitly reported via `Result`.
- `Serializable`: Encodes a structured header or payload back into a writable byte buffer (`&mut [u8]`), supporting outbound packet construction.
- `Header`: Provides accessor methods for reading and updating specific fields (e.g., source port, sequence number, flags) without exposing raw offsets or memory layout.

This design promotes a clear separation of concerns. Each protocol layer operates generically over trait bounds and does not require knowledge of the internal structure of the layers it encapsulates. Protocols are implemented as independent modules responsible for parsing their respective layers and delegating to the next. This results in a composable pipeline, where layers can be stacked dynamically at runtime or statically at compile time, depending on the application context. For example:

Listing 1: Layered Abstractions

```
let eth = EtHeader::parse(frame)?;
let ip = Ip4Header::parse(eth.payload())?;
let udp = UdpHeader::parse(ip.payload())?;
let app = parse_dns(udp.payload())?;
```

This compositional parsing approach eliminates the need for manual offset calculations and nested casting logic, which are common sources of bugs in C-based stacks. Rust's type system provides static guarantees on header length and lifetime validity, thereby reducing the need for runtime checks and enhancing overall system robustness.

Adding new protocols — such as IPv6, ICMPv6, or experimental transport layers like QUIC — requires only the implementation of the relevant traits for the new header types. Since each layer adheres to a common trait interface, integration with the rest of the stack requires minimal changes. Furthermore, the decoupling of parsing and serialization logic from control plane logic allows new protocols to be unit-tested and fuzzed independently.

**Lessons Learned**

Manual offset handling and nested structs led to alignment issues and borrow checker conflicts. Adopting a trait-based design clarified parsing logic and improved safety, highlighting the importance of embracing Rust's type system over low-level C idioms.

*4.1.4. From `pbuf` chains to ownership-aware packet buffers*

*LwIP* uses `pbuf` chains to enable reference-counted buffer reuse, but this design depends on manual memory management and often leads to complex lifetime issues and potential memory safety violations such as use-after-free or double free.

To address these problems, we introduce a `PacketBuffer` abstraction built atop Rust's `Vec<u8>`, designed with controlled headroom/tailroom and strict borrowing semantics. Unlike the shared, reference-counted model in `pbuf`, our approach ensures memory safety through explicit ownership transfer and runtime visibility of resource state:

- Ownership of each buffer is explicitly transferred across layers, eliminating shared mutable state.
- Zero-copy access is supported via scoped borrowing without violating aliasing rules.
- Buffers are wrapped in `Option<Buf>` to explicitly track their validity; once a buffer is consumed or released, it is set to `None`, preventing accidental reuse or double-free through runtime checks.

This ownership-aware model harmonizes with Rust's lifetime system, enables safe and efficient buffer reuse, and greatly simplifies debugging and resource cleanup compared to `pbuf`'s distributed and implicit reference management.

*4.1.5. Handling cyclic references and lifetimes in TCP connections*

TCP's design introduces intrinsic bidirectional references: a TCP control block (PCB) often needs to access its parent network interface, while the interface maintains a list of active PCBs. In C, this is trivially implemented via raw pointers—but in Rust, such cycles are disallowed by the ownership system.

To resolve this, we apply the following strategies:

- Use `Rc<RefCell<T>>` and `Weak<T>` in single-threaded contexts to support non-owning backreferences while avoiding reference cycles.
- Redesign data flows to pass context (e.g., parent interface) explicitly into function calls rather than embedding long-lived references.
- Decompose monolithic PCB structs into smaller state holders (e.g., `TimerState`, `RetransmissionState`) to reduce the need for shared state across layers.

This experience confirms the benefits of acyclic ownership trees and explicit context passing. Although this restructuring increases verbosity, it dramatically improves reasoning about lifetimes, enables safe concurrent access patterns, and eliminates entire classes of dangling pointer bugs found in *LwIP*.

**Lessons Learned**

Adapting *LwIP*'s pointer-centric design to Rust exposed fundamental conflicts with ownership and lifetime constraints. Transitioning to ownership-based buffers, acyclic references, and explicit context passing eliminated key memory risks. Building safe systems in Rust requires rethinking C idioms, not reusing them.

### 4.2. Memory safety through ownership-based resource management

The *LwIP* implementation extensively used raw pointers, manual memory management, and custom reference counting. Protocol components such as `pbuf`, `tcp_pcb`, and `netif` are passed across layers using unsafe pointers, with lifetimes managed manually. This design introduces serious risks that are notoriously difficult to debug in large, layered systems.

In our Rust-based reimplementation, we replace this model with a strictly ownership-driven design. Leveraging Rust's ownership and borrowing semantics, we achieve memory safety without relying on runtime garbage collection or reference counting.

Instead of relying on global or linked-list-based control blocks, all protocol states are stored in containers with clear ownership. For example, TCP sockets are held in a `HashMap<SocketId, TcpSocket>` owned by the `TcpManager`. Each `TcpSocket` owns its internal buffers, timers, and control structures.

This design ensures that memory is automatically reclaimed when the owning manager is dropped, and that no part of the system can access invalidated memory.

The `pbuf` system in *LwIP* employs shared, reference-counted buffer chains to support zero-copy packet processing. In LwRustIP, we replace this with a single-owner `PacketBuffer` abstraction built on `Vec<u8>` and explicit offset metadata. Higher-level components operate on borrowed views (`&[u8]` or `BufRef<'a>`) for parsing or dispatch, avoiding any memory copying.

This model eliminates aliasing and race conditions by enforcing Rust's strict borrowing rules at compile time. Since each buffer has a single owner, there is no need for `Rc`, `Arc`, or manual reference counting. Packet reassembly and retransmission scenarios are handled with controlled cloning, and all access paths are validated through lifetimes.

To replace *LwIP*'s custom memory pools (e.g., `memp_malloc`, `memp_free`), we introduced arena-style and slot-based allocators with safe handle types. For example, TCP control blocks are managed using a `SlotMap` indexed by unique IDs, ensuring that no stale or dangling handles can be reused.

Listing 2: TcpPool

```
type TcpSocketHandle = u32;

struct TcpPool {
    slots: SlotMap<TcpSocketHandle, TcpSocket
        >,
}
```

These allocators provide the following advantages:

- Enforcement of compile-time memory limits;
- Automatic resource cleanup through `Drop`;
- Lifetime-based access control without unsafe code.

Compared to the global, opaque allocators in *LwIP*, this design improves modularity, testability, and robustness under concurrent access.

- Use `Rc<RefCell<T>>` and `Weak<T>` in single-threaded contexts to support non-owning backreferences while avoiding reference cycles.
- Redesign data flows to pass context (e.g., parent interface) explicitly into function calls rather than embedding long-lived references.
- Decompose monolithic PCB structs into smaller state holders (e.g., `TimerState`, `RetransmissionState`) to reduce the need for shared state across layers.

This experience confirms the benefits of acyclic ownership trees and explicit context passing. Although this restructuring increases verbosity, it dramatically improves reasoning about lifetimes, enables safe concurrent access patterns, and eliminates entire classes of dangling pointer bugs found in *LwIP*.

**Lessons Learned**

We learned to decompose shared state into explicitly owned components, rely on scoped borrowing, and restructure logic to align with ownership boundaries. This approach not only eliminated memory bugs but also enhanced modularity and long-term maintainability, reinforcing ownership as a core design discipline in Rust.

### 4.3. Engineering experience and lessons learned

The reimplementation of *LwIP* in Rust is not merely a syntactic translation, but a comprehensive architectural transformation. The process required deep rethinking of system design principles, rigorous engineering effort, and careful reconciliation between performance and safety. It reveals both the strengths of Rust as a systems programming language and the practical challenges of migrating from legacy C codebases.

The migration effort spanned approximately eight months and proceeded through incremental prototyping. Rather than attempting a direct file-by-file translation, we adopted a bottom-up, component-driven approach. We started with a minimal ICMP echo server and gradually integrated protocol support, memory management, and interface abstraction. Each step was validated through targeted unit tests and performance benchmarks.

The majority of the engineering time — approximately 80% — was spent redesigning critical subsystems, including the TCP/IP stack, buffer management, and memory allocators, to comply with Rust's ownership and borrowing model. The remaining time was dedicated to integration testing, real-world workload simulation, and performance tuning.

Several challenges emerged during this process:

- **Semantic Misalignment**: Rust's strict ownership and aliasing rules necessitated a complete rethinking of how shared buffers and packet state were managed. Designs that were straightforward in C — such as reference-counted pbuf chains — proved fundamentally incompatible with Rust's linear type system.
- **Compiler Constraints as Design Feedback**: The Rust borrow checker exposed implicit bugs and race conditions latent in the original C codebase. These early failures, though frustrating, guided the transition toward more robust and modular abstractions.
- **Ecosystem Limitations**: Although Rust's ecosystem continues to mature, certain low-level capabilities — particularly for DMA-safe zero-copy buffers and hardware abstraction — still require manual implementation. This necessitated custom trait hierarchies and careful encapsulation of unsafe code.

Through this experience, we distilled several key lessons that may inform future efforts to develop or migrate system-level software in Rust:

**Table 2**
Hardware testing environment.

| Device | CPU | RAM | Network interfaces |
|---|---|---|---|
| Allwinner T507 | ARM Cortex-A53 @1.5 GHz (quad-core) | 1GB | 1× 1000 Mbps, × 100 Mbps |
| Raspberry Pi 3B | ARM Cortex-A53 @1.24 GHz (quad-core) | 1GB | 1× 100 Mbps |

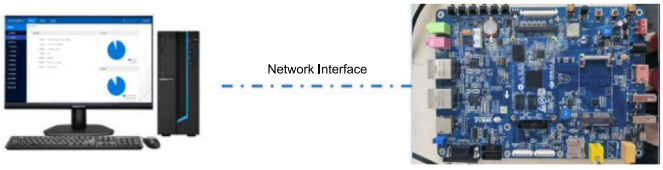


**Fig. 1.** Allwinner T507 development board.

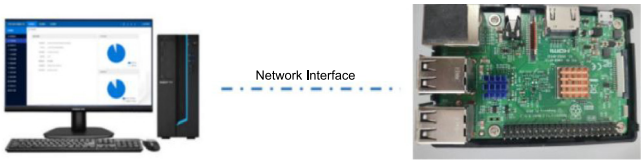


**Fig. 2.** Raspberry Pi 3B.

- **Design Around Ownership, Not Just Performance**: Initial concerns that Rust's ownership model might hinder performance proved unfounded. In fact, rearchitecting to minimize shared mutable state often led to improved concurrency and cache locality.
- **Start Minimal and Integrate Incrementally**: Building a working prototype provided a concrete baseline for testing assumptions, evaluating architectural trade-offs, and guiding subsystem integration.
- **Rust Is Not "C with Types"**: Effective Rust development requires embracing new paradigms. Attempting to directly port imperative C idioms often leads to friction with the type system and borrow checker.
- **Early Testing Is Critical**: Incorporating fuzz testing and property-based testing from the outset helped uncover rare edge cases and protocol violations that would have escaped traditional unit tests.
- **Documentation and Tooling Matter**: Internal documentation, test harnesses, and consistent module boundaries significantly reduced onboarding time and improved debugging across the team.

This experience demonstrates that, although migrating to Rust requires substantial upfront engineering effort, it delivers long-term benefits in safety, maintainability, and testability. We believe these practices and insights provide a practical foundation for future Rust-based systems, particularly in domains that demand reliability, concurrency, and low-level performance.

## 5. Evaluation

### 5.1. Testing environment

The evaluation leverages two embedded platforms: the Allwinner T507 development board and the Raspberry Pi 3B. Their hardware specifications are summarized in Table 2.

As shown in Figs. 1 and 2, the Allwinner T507 and Raspberry Pi 3B serve as primary testbeds, connected to a host PC via Ethernet for performance benchmarking.

The evaluation employs Wireshark and Iperf3 for packet analysis and performance measurement. Wireshark, a widely-used open-source protocol analyzer, captures and decodes network traffic in real time, enabling detailed inspection of packet headers and payloads. Iperf3 provides standardized network throughput testing, supporting bidirectional bandwidth measurement between client–server pairs.

The host PC runs Wireshark and Iperf3, while the embedded boards execute the proposed protocol stack. Communication occurs over wired Ethernet, with the protocol stack configured with a static IP address (172.16.0.88) and the host PC set to 172.16.0.200. Wireshark validates protocol compliance and packet integrity, while Iperf3 quantifies bandwidth.

**Table 3**
Functional test results.

| Test case | Protocol | Result |
|---|---|---|
| ICMP ping | ICMP | Passed |
| ARP resolution | ARP | Passed |
| TCP communication | TCP | Passed |
| UDP communication | UDP | Passed |

**Table 4**
Throughput across packet sizes (Mbps).

| TCP/IP stack | 1B | 10B | 100B | 1KB | 1MB |
|---|---|---|---|---|---|
| LwRustIP | 4.868 | 48.805 | 391.3 | 945.4 | 948.0 |
| *LwIP* | 4.872 | 48.785 | 393.05 | 944.05 | 944.35 |
| Linux | 4.869 | 48.765 | 429.05 | 936.7 | 937.15 |

### 5.2. Functional testing

We conducted comprehensive functional tests to verify the correctness of core networking components, including ARP, ICMP, TCP, and UDP. The protocol stack was deployed on an embedded device and connected to a host PC via Ethernet. Each protocol was tested using standard tools and representative communication scenarios. ICMP and ARP functionality was verified using ping, confirming proper address resolution and echo-reply handling. TCP was tested in both client and server modes to ensure correct connection establishment and data transmission. UDP functionality was validated by exchanging datagrams between the host and the stack, confirming accurate header construction and payload delivery.

All tests confirmed protocol compliance and correct behavior under typical usage conditions. The results are summarized in Table 3, with all cases passing successfully.

### 5.3. Performance evaluation

#### 5.3.1. Overall performance

Performance metrics for LwRustIP, *LwIP*, and the Linux kernel stack were collected on the Allwinner T507 development board under both Gigabit and 100 Mbps network configurations.

As Table 4, LwRustIP's throughput across varying packet sizes (1B–1MB) was benchmarked against *LwIP* and Linux. LwRustIP demonstrates stable performance for both small and large packets, achieving near-line rate throughput (> 945 Mbps) for 1KB+ packets. While *LwIP* shows comparable performance for small packets, LwRustIP outperforms both competitors in large-packet scenarios.

**Table 5**
Bandwidth performance comparison (Mbps).

| TCP/IP stack | GbE receive | GbE transmit | 100M receive | 100M transmit |
|---|---|---|---|---|
| LwRustIP | 945.42 | 725.04 | 94.69 | 95.00 |
| *LwIP* | 948.79 | 694.53 | 94.95 | 95.00 |
| Linux | 941.38 | 828.00 | 94.15 | 94.59 |

**Table 6**
Latency performance (ms).

| TCP/IP stack | GbE latency | 100M latency |
|---|---|---|
| LwRustIP | 1.36 | 0.653 |
| *LwIP* | 1.27 | 0.702 |
| Linux | 1.447 | 0.736 |

Table 5 presents the aggregate throughput performance of three TCP/IP stacks — LwRustIP, *LwIP*, and Linux — under both Gigabit Ethernet (GbE) and 100 Mbps conditions. The results demonstrate that LwRustIP achieves comparable performance to *LwIP* and the native Linux stack. Specifically, LwRustIP attains 945.42 Mbps in GbE receive tests and 725.04 Mbps in GbE transmit, closely matching *LwIP*'s 948.79 Mbps and 694.53 Mbps respectively. On 100 Mbps links, LwRustIP reaches near line-rate performance at 94.69 Mbps and 95.00 Mbps. These results show that LwRustIP, despite introducing strict memory safety mechanisms through Rust's ownership system, does not impose any significant bandwidth overhead and remains competitive with hand-optimized C implementations.

Table 6 compares the end-to-end latency performance of LwRustIP, *LwIP*, and the native Linux TCP/IP stack under both Gigabit Ethernet (GbE) and 100 Mbps Ethernet environments. Under GbE conditions, LwRustIP records a latency of 1.36 ms, which is slightly higher than *LwIP*'s 1.27 ms but lower than Linux's 1.447 ms. In the 100 Mbps scenario, LwRustIP achieves the lowest latency of 0.653 ms, outperforming both *LwIP* (0.702 ms) and Linux (0.736 ms). These results indicate that the memory safety guarantees and abstractions in Rust do not introduce notable latency penalties.

Overall, the results confirm that LwRustIP delivers memory-safe network stack semantics without compromising on performance, validating that systems-level safety and efficiency can coexist through Rust's design principles.

#### 5.3.2. Performance trade-off

To clarify the trade-off between memory safety and performance, we conducted fine-grained microbenchmarking of each processing stage using both safe and selectively optimized unsafe Rust implementations. Table 7 reports the average latency per stage under both modes. Results show that while safe Rust introduces minimal overhead in certain stages (e.g., TCP processing and output), the performance gap remains narrow and within acceptable bounds.

Notably, the final design of LwRustIP ensures that memory safety is preserved without incurring additional runtime cost. As evidenced in previous bandwidth and latency tests, the safe-by-default implementation of LwRustIP achieves performance on par with *LwIP* and Linux, confirming that Rust's safety guarantees can coexist with systems-level efficiency.

### 5.4. Memory safety analysis

#### 5.4.1. Buffer overflow

Buffer overflow vulnerabilities — common in traditional network stacks such as *LwIP*(e.g., CVE-2024-7490) and uIP (e.g., CVE-2020-17437) — occur when data exceeds the bounds of preallocated buffers, potentially enabling arbitrary code execution or denial-of-service attacks. LwRustIP eliminates such risks by enforcing compile-time boundary checks on all buffer operations. Each protocol layer validates packet lengths against buffer capacities during parsing to ensure memory safety.

As illustrated in Listing 3, when processing an incoming packet, the stack compares the length specified in the packet header with the actual buffer size and immediately aborts the operation if an overflow attempt is detected.

This mechanism is demonstrated in a test case with a 10-bytes payload: accessing payload[2] correctly returns 0x22, while accessing payload[11] triggers a runtime panic, thereby preventing out-of-bounds memory access.

Listing 3: Overflow test code

```
let payload: Vec<u8> = vec![
    0x58, 0x11, 0x22, 0x82, 0x48,
    0x29, 0x8e, 0x3c, 0x47, 0x8a
];
let a: u8 = payload[2];
println!("the value of a is {}", a);
let b: u8 = payload[11];
// will panic
println!("the value of b is {}", b);
```

#### 5.4.2. Null/dangling pointer

Null pointer dereferences and dangling pointers are fundamentally eliminated in LwRustIP through Rust's ownership and lifetime system. All references are guaranteed to be initialized before use, and strict lifetime annotations enforce proper scoping and prevent use-after-free errors at compile time.

As demonstrated in Listing 4, accessing an uninitialized object results in a compile-time error. Here, the variable 'packet' is declared but never initialized, and any attempt to access its internal field leads to a compilation failure.

Similarly, Listing 5 illustrates how Rust prevents dangling pointers. Once the memory owned by the payload vector is freed, any subsequent access to a reference derived from it is statically rejected by the compiler, thereby preventing use-after-free vulnerabilities.

Listing 4: Uninitialized test code

```
pub struct TcpPacket<'a> {
    buffer: &'a mut [u8]
}
fn main() {
    // Declaration without initialization
    let packet: TcpPacket;
    println!("The tcp packet is {:?}", packet.
        buffer);
}
```

Listing 5: Dangling Pointer test code

```
let payload: Vec<u8> = vec![
    0x58, 0x11, 0x22, 0x82, 0x48,
    0x29, 0x8e, 0x3c, 0x47, 0x8a
];
println!("The tcp packet is {:?}", packet.
    buffer);
drop(payload);
// compile error use after free
println!("The tcp packet is {:?} after free",
    packet.buffer);
```

**Table 7**
Per-stage processing latency (ns): safe vs. unsafe rust.

| Metric | IP processing | TCP processing | TCP segment creation | TCP segment output | IP output | Ethernet output |
|---|---|---|---|---|---|---|
| Safe (avg) | 1150 | 250 | 875 | 1930 | 288 | 190 |
| Unsafe (avg) | 1147 | 215 | 850 | 1885 | 270 | 175 |

*5.4.3. Double-free and memory leak*

Double-free errors — often arising from manual deallocation or incorrect memory state tracking in systems languages like C — are statically prevented in LwRustIP. This is achieved through Rust's ownership model, which ensures that each heap-allocated resource has a unique owner. When the owner goes out of scope, the resource is deallocated exactly once. The compiler prohibits copying or moving ownership without explicit annotation, thereby making double frees impossible by design.

Listing 6: Lifetime test code

```
{
    let payload: Vec<u8> = vec![
        0x58, 0x11, 0x22, 0x82, 0x48,
        0x29, 0x8e, 0x3c, 0x47, 0x8a
    ];
}// 'payload' is dropped here, its lifetime
    ends
println!("The tcp packet is {:?}", packet.
    buffer);
```

Memory leaks are also significantly mitigated. Stack-allocated and heap-allocated variables are automatically dropped when they fall out of scope. Any attempt to use a reference (e.g., a packet or buffer) outside its lexical lifetime is rejected at compile time. This guarantees that no memory is inadvertently left unreleased or misused.

Listing 6 provides a concrete example: a vector payload is created inside a block scope. After the block ends, its memory is automatically freed. Any subsequent use of a value referencing payload triggers a compile-time error, preventing undefined behavior.

## 6. Conclusion

In this work, we present LwRustIP, a memory-safe, efficient, and *LwIP*-compatible embedded network stack reimplemented in Rust. Our motivation arises from the well-known security limitations of C-based network stacks in embedded systems, where memory-unsafe operations — such as buffer overflows, dangling pointers, and use-after-free — remain persistent threats. By leveraging Rust's ownership semantics, zero-copy buffer designs, and lock-free memory pools, LwRustIP demonstrates that memory safety and performance are not mutually exclusive, even in resource-constrained environments.

Our implementation required substantial architectural refactoring, transitioning from *LwIP*'s monolithic, shared-state design to a modular, ownership-driven architecture. We systematically decomposed protocol layers into self-contained Rust modules, employed trait-based parsing for composability, and adopted idiomatic Rust constructs such as once_cell, Rc<RefCell>, and zero-copy PacketBuffer abstractions. This transformation demanded a fundamental rethinking of legacy C patterns and revealed both the challenges and advantages of Rust's safety model.

Through this migration, we gained valuable insights into the practical realities of retrofitting memory safety into low-level systems. Many of Rust's restrictions — initially perceived as barriers — ultimately enhanced modularity, testability, and performance predictability. Our iterative, test-driven development process helped uncover subtle logic flaws in the original design, while fuzz testing proved invaluable for validating correctness beyond conventional unit tests.

Experimental results on ARM-based embedded platforms show that LwRustIP retains compatibility and performance comparable to the original *LwIP* stack, while achieving full memory safety without measurable overhead. These findings affirm that Rust is not only viable but often preferable for building secure and efficient embedded system components.

We hope this experience report offers actionable insights for developers aiming to migrate legacy C codebases to Rust. In particular, we emphasize the value of architectural rethinking, modular decomposition, and incremental integration when pursuing memory safety in low-level systems. As embedded software becomes increasingly security-critical, we believe Rust will play a pivotal role in shaping the next generation of trustworthy systems.

## CRediT authorship contribution statement

**Guangyong Shang:** Writing – review & editing, Writing – original draft, Validation, Software, Resources, Project administration, Methodology, Investigation, Data curation, Conceptualization. **Guangpeng Qi:** Supervision, Project administration, Formal analysis, Data curation, Conceptualization. **Jianing Ren:** Methodology, Investigation, Formal analysis, Data curation. **Xianqi Jin:** Writing – review & editing, Visualization, Supervision, Data curation, Conceptualization. **Wanjiang Shen:** Validation, Software, Project administration, Methodology, Investigation. **Junchao Li:** Writing – review & editing, Software, Resources, Investigation, Data curation. **Runyu Pan:** Writing – review & editing, Writing – original draft, Validation, Software, Data curation, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This work was partially supported by National Key Research and Development Program of China (2022YFB4502001), National Natural Science Foundation of China (62402291, 62302265, U23A20332), and Shandong Province Natural Science Foundation (ZR2023QF172, 2024HWYQ-020).

## References

[1] F. Meneghello, M. Calore, D. Zucchetto, M. Polese, A. Zanella, IoT: Internet of threats? A survey of practical security vulnerabilities in real IoT devices, IEEE Internet Things J. 6 (5) (2019) 8182–8201, http://dx.doi.org/10.1109/JIOT.2019.2935189.

[2] U. Lindqvist, P.G. Neumann, The future of the internet of things, Commun. ACM 60 (2) (2017) 26–30, http://dx.doi.org/10.15242/IJCCIE.AE0417133.

[3] PurpleSec, 9 common types of malware (& how to prevent them), 2024, (Accessed 20 April 2025), https://purplesec.us/learn/common-malware-types/.

[4] Armis Centrix, 11 zero day vulnerabilities impacting billions of mission-critical devices, 2020, (Accessed 20 April 2025), https://www.armis.com/research/urgent-11/.

[5] Vedere Labs, AMNESIA:33, 2020, (Accessed 20 April 2025), https://www.forescout.com/research-labs/amnesia33/.
[6] U.S. Department of Homeland Security (DHS) Cybersecurity, Infrastructure Security Agency (CISA) , CVE: Common vulnerabilities and exposures, 2025, (Accessed 10 April 2025), https://cve.mitre.org/.
[7] P.C. Amusuo, R.A.C. Méndez, Z. Xu, A. Machiry, J.C. Davis, Systematically detecting packet validation vulnerabilities in embedded network stacks, in: 2023 38th IEEE/ACM International Conference on Automated Software Engineering, ASE, IEEE, 2023, pp. 926–938, http://dx.doi.org/10.1109/ASE56229.2023.00095.
[8] W. Wang, Finding and exploiting vulnerabilities in embedded TCP/IP stacks, 2021, URL http://essay.utwente.nl/88684/.
[9] Tock, Tock embedded operating system, 2014, (Accessed 13 April 2025), https://tockos.org/.
[10] Redox, The redox operating system, 2015, (Accessed 13 April 2025), https://tockos.org/.
[11] V. Valyaeff, Drone | an embedded operating system for writing real-time applications in rust, 2025, (Accessed 13 April 2025), https://www.drone-os.com/.
[12] K. Boos, N. Liyanage, R. Ijaz, L. Zhong, Theseus: an experiment in operating system structure and state management, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020, pp. 1–19, URL https://www.usenix.org/conference/osdi20/presentation/boos.
[13] D. Nieuwenhuis, Smoltcp, 2025, (Accessed 11 April 2025), https://github.com/smoltcp-rs/smoltcp.
[14] U. Lilleengen, Embassy: Modern embedded framework, using rust and async, 2025, (Accessed 11 April 2025), https://github.com/embassy-rs/embassy.
[15] R. Javaux, Rusty: A light-weight, user-space, event-driven, highly-scalable, TCP/IP stack using Tilera's mPIPE API, 2025, (Accessed 11 April 2025), https://github.com/RaphaelJ/rusty.
[16] polyelectronics, The rust programming language for embedded systems, 2024, (Accessed 25 April 2025), https://polyelectronics.us/the-rust-programming-language-for-embedded-systems/.
[17] A. Sharma, S. Sharma, S. Torres-Arias, A. Machiry, Rust for embedded systems: current state, challenges and open problems, 2023, arXiv preprint arXiv:2311.05063.
[18] Bytecode Alliance, Rustix: Safe rust bindings to POSIX-ish APIs, 2025, (Accessed 28 April 2025), https://github.com/bytecodealliance/rustix.
[19] GNOME, Librsvg: A small library to render scalable vector graphics, 2016, (Accessed 28 April 2025), https://github.com/GNOME/librsvg.
[20] H.W. Andreea Florescu, Secure and fast microvms for serverless computing., 2025, (Accessed 28 April 2025), https://github.com/firecracker-microvm/firecracker.
[21] R. Jung, J.-H. Jourdan, R. Krebbers, D. Dreyer, RustBelt: Securing the foundations of the rust programming language, Proc. the ACM Program. Lang. 2 (POPL) (2017) 1–34, http://dx.doi.org/10.1145/3158154.
[22] R. Jung, UCG - rust's unsafe code guidelines, 2025, (Accessed 26 April 2025), https://github.com/rust-lang/unsafe-code-guidelines.
[23] Verification Infrastructure for Permission-based Reasoning, Prusti: A static verifier for rust, 2025, (Accessed 26 April 2025), https://github.com/viperproject/prusti-dev.
[24] M. Tautschnig, Kani rust verifier, 2025, (Accessed 26 April 2025), https://github.com/model-checking/kani.
[25] Creusot, Creusot is a deductive verifier for rust code, 2025, (Accessed 26 April 2025), https://github.com/creusot-rs/creusot.
[26] E. Labs, An abstract interpreter for the rust compiler's mid-level intermediate representation, 2025, (Accessed 26 April 2025), https://github.com/endorlabs/MIRAI.
[27] R.C. Seacord, The CERT C secure coding standard, Pearson Education, 2008, URL https://wiki.sei.cmu.edu/confluence/display/c/SEI+CERT+C+Coding+Standard.
[28] CodeSecure, Joint strike force, 2025, (Accessed 28 April 2025), https://support.codesecure.com/hc/en-us/articles/15448705954578-Joint-Strike-Force.
[29] MISRA, MISRA c/c++, 2021, (Accessed 28 April 2025), https://misra.org.uk/.
[30] P. Anderson, Coding standards for high-confidence embedded systems, in: MILCOM 2008 - 2008 IEEE Military Communications Conference, 2008, pp. 1–7, http://dx.doi.org/10.1109/MILCOM.2008.4753206.
[31] A. Dunkels, lwIP - A lightweight TCP/IP stack, 2002, (Accessed 29 April 2025), https://savannah.nongnu.org/projects/lwip/.
[32] A. Dunkels, uIP-a free small TCP/IP stack, 2002, URL https://www.dunkels.com/adam/download/uip-doc-0.6.pdf.
[33] F.C. Andrey Butok, fNET - embedded TCP/IP stack, 2005, (Accessed 29 April 2025), https://fnet.sourceforge.io/.
[34] CVE, A buffer overflow vulnerability in lwIP, 2020, (Accessed 30 April 2025), https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-22283.
[35] CVE, Double free vulnerability in virtualsquare picoTCP, 2021, (Accessed 30 April 2025), https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-33304.
[36] CVE, VxWorks IPNET CVE has a NULL pointer dereference, 2020, (Accessed 30 April 2025), https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2020-10664.
[37] CVE, Memory leak in linux TCP/IP stack, 2024, (Accessed 30 April 2025), https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2024-57841.
[38] RustPython, 2025, (Accessed 16 July 2025), https://github.com/RustPython/RustPython.